
Arduino Project

Line Following Robot with Object Detection or
Contact-less Delivery System

Group 19

1)Dajr Alfred

2)Mohammed Nauman Shariff

3)Gordon Oboh

03/10/2022

Contents

Arduino Project	1
Contents	2
1. Introduction	3
Brief Description of the scenario	3
1.1 Problem Objective and it's Solution	3
2. Methodology	5
2.1 Circuit Diagram and explanation	5
2.2 Pin Representation	6
3. Hardware Session	7
3.1 Arduino Nano	8
3.2 Ultrasonic Sensor	9
3.3 Pololu IR sensor Array	10
4. Software Session	12
4.1 Code:	12
5. Result and Conclusion	14

1. Introduction

Brief Description of the scenario

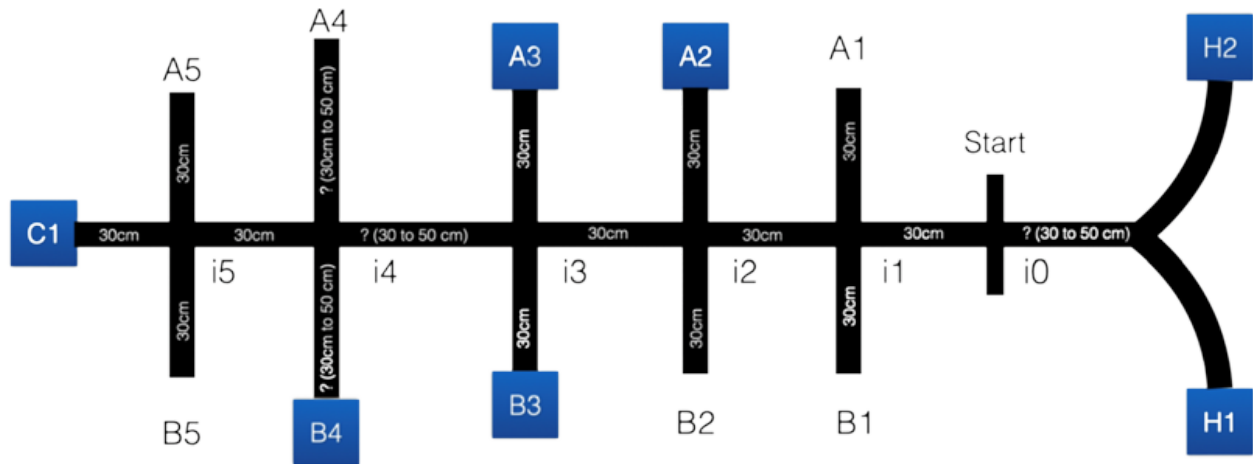


Figure 1: The map of the parking lot

1.1 Problem Objective and it's Solution

With the emergence of COVID 19 day-to-day activities have been impeded. Due to this unusual situation, the need for a contact less delivery system arose. The delivery system will allow a covid test kit provider to deliver the test kits at desired locations within the parking lot. Figure 1 is an outline of the track the robot will have to follow. There are two home locations (H1 and H2) from where the robot starts out on the track and it has to cross 6 intersections (i0 to i5) with up to 11 possible delivery points to complete its mission.

Our implementation of the line following robot uses an eight-channel digital IR Reflectance sensor to follow the line and detect intersections. If the leftmost IR sensor detects black tape the Arduino compensates and tries to put the robot back on track by moving towards the right, if the rightmost IR sensor detects black tape the arduino compensates for the error by moving towards the left. Using this mechanism, the bot follows the direction of the tape and needs to stop whenever it finds the intersections. For this, we could command the Arduino in such a way that, if it detects black tape on both sides, it needs to stop.

When intersections are detected both LEDs are flashed, 3 ultrasonic sensors are used to scan for possible objects on the track, if an object is detected on the left side, the LED on the left flashes and if the object is detected on the right side, the LED on the right flashes. If an object is detected

on both sides, then the robot moves towards the right object first and then comes back to its initial position, later goes towards the left object. After reaching the respective positions, the robot is implemented to stop 7cm before touching the object and the respective LEDs are flashed. 7cm was picked because this prevented the robot from touching the object when turning on its wheel axis to return back to the main track, we programmed it to go back a few steps backward and then rotate and find the track. This process continues until the robot gets to i5 intersection where it does a final check and stops.

2. Methodology

2.1 Circuit Diagram and explanation

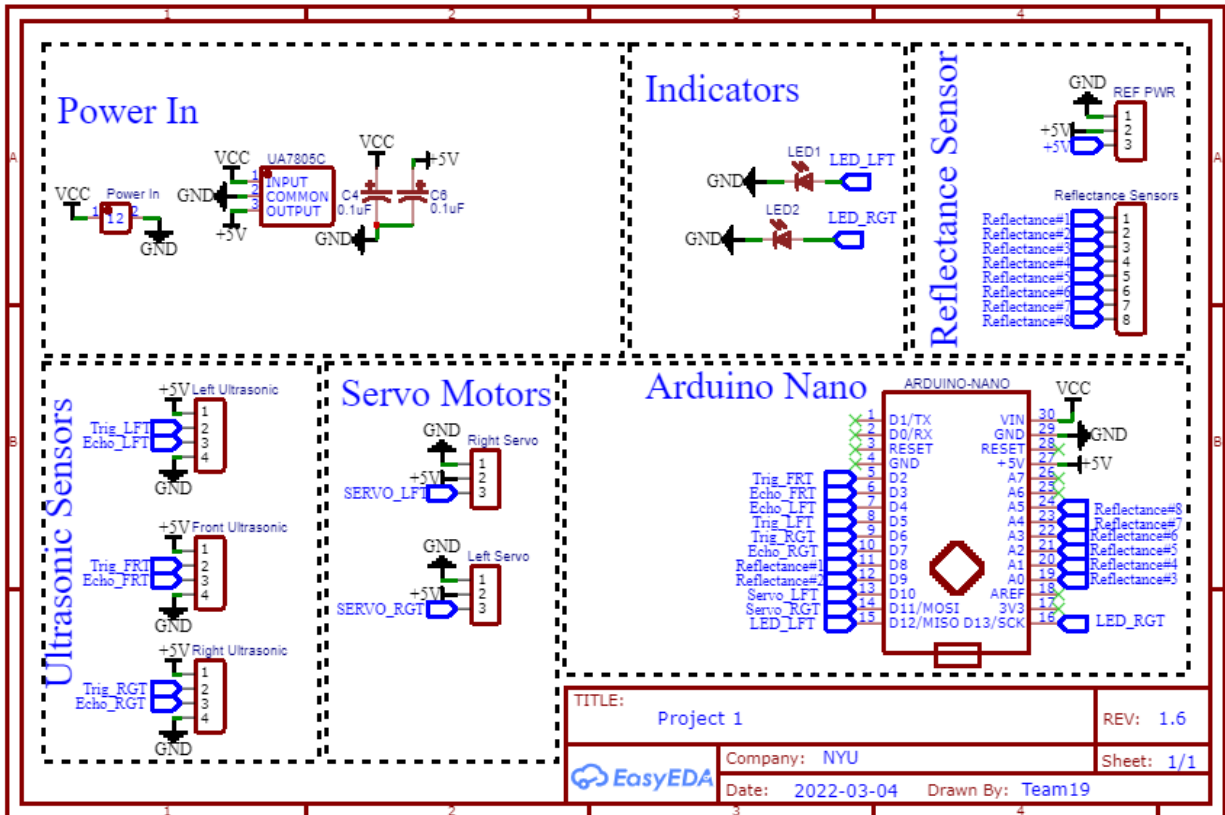


Figure 2: Circuit Implementation based on the Arduino Code

The above Circuit Diagram is implemented to show the commands we have implemented in our Arduino Code using EasyEDA Software program.

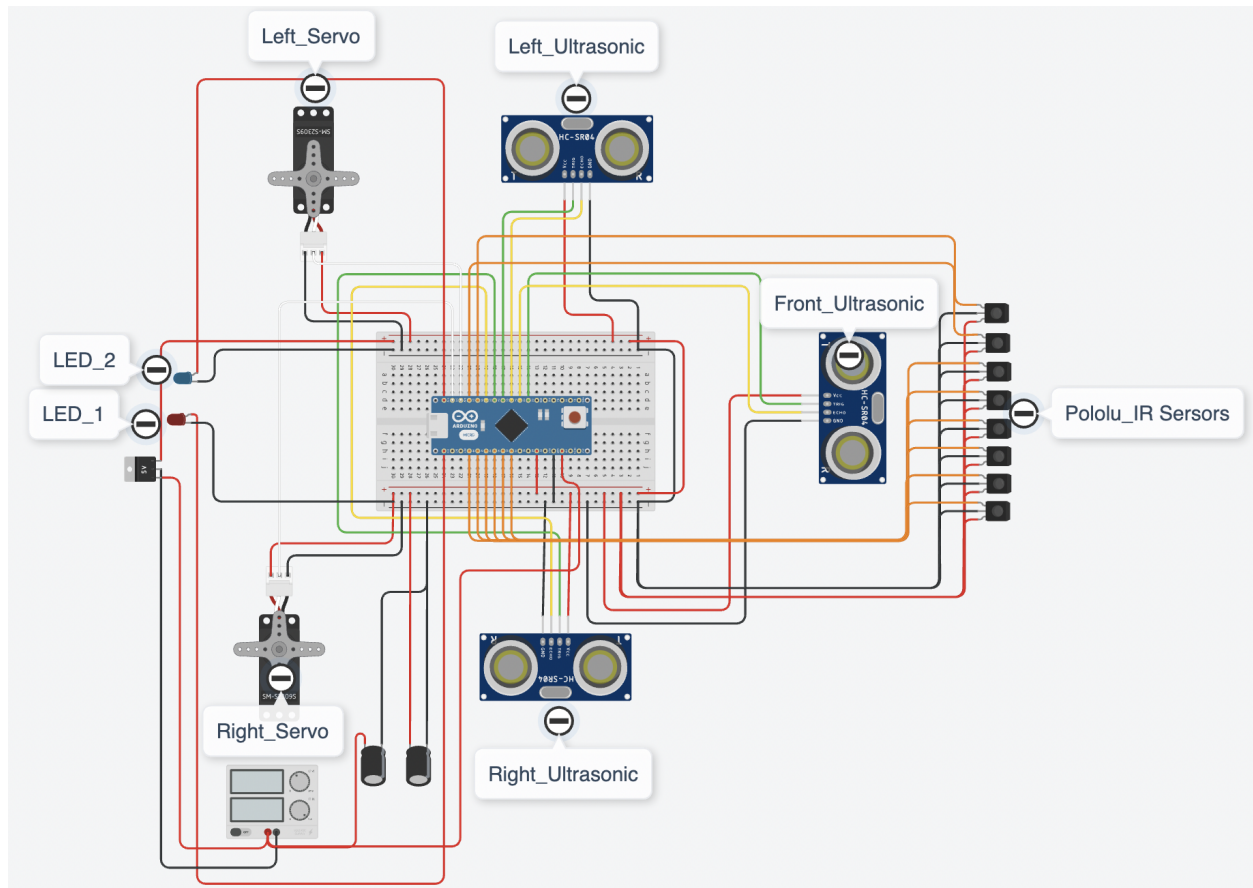


Figure 3: Pictorial representation of the Circuit Diagram and it's connections

2.2 Pin Representation

1) Digital Pins

- Pin 2 - Trigger of Ultrasonic sensor 1
- Pin 3 - Echo of Ultrasonic sensor 1
- Pin 4 - Echo of Ultrasonic sensor 2
- Pin 5 - Trigger of Ultrasonic sensor 2
- Pin 6 - Trigger of Ultrasonic sensor 3
- Pin 7 - Echo of Ultrasonic sensor 3
- Pin 8 - Signal for Pololu IR Sensor 1
- Pin 9 - Signal for Pololu IR Sensor 2
- Pin 10 - Signal for Left Servo Motor
- Pin 11 - Signal for Right Servo Motor
- Pin 12 - Left LED indicator
- Pin 13 - Right LED indicator

2) Analog Pins

- Pin A0 - Signal for Pololu IR Sensor 3
- Pin A1 - Signal for Pololu IR Sensor 4
- Pin A2 - Signal for Pololu IR Sensor 5
- Pin A3 - Signal for Pololu IR Sensor 6
- Pin A4 - Signal for Pololu IR Sensor 7
- Pin A5 - Signal for Pololu IR Sensor 8
- Pin A6 - No connection
- Pin A7 - No connection

Figure 3 shows the pictorial representation of the circuit diagram, this was accomplished using the Tinkercad software. An Arduino Nano was used due to space constraints and we initially thought since it had more pins it would be an advantage. The Battery supplies a total of 12V and this voltage is sent to a voltage regulator and Stepped down to 5V. The capacitors are used as a filter to reduce ripples from the voltage regulator. The 12V power supply is also sent to Vin on the Arduino Nano. The 5V Supply on the Arduino Nano is used to power all the electronics. The explanation for each representation is done in the Hardware session.

3. Hardware Session

3.1 Arduino Nano

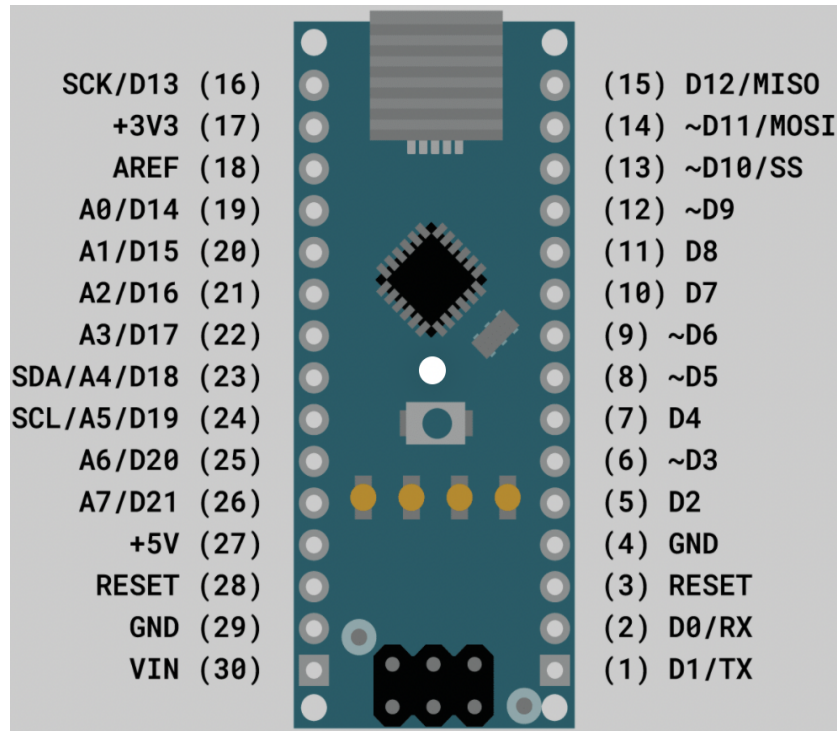


Figure 4: Arduino Nano Microcontroller

The open-source Arduino Nano uses 6-20V unregulated power supply and is powered via the Mini-B USB connector. It has the same microcontroller as the Arduino Uno which is ATmega328. It has 3 different types of memory.

- Flash memory - 32kb
- EEPROM - 1kb
- SRAM - 2kb

All the 14 digital pins on the board can be used as inputs or outputs with the functionality `pinMode()`, `digitalWrite()`, and `digitalRead()`. The 8 Analog pins measuring from ground to 5V have 10 bits of resolution.

Pin representation:

Serial Pin : 0 - RX and 1 - TX

They are used to receive and transmit serial data.

PWM : 3,5,6,9,10 and 11

They have the same PWM pins as compared to the Arduino Uno which provides 8-bit output using `analogWrite()` functionality.

SPI : Serial Peripheral Interface with Pin 10(SS), 11(MOSI), 12(MISO) and 13(SCK).

Ext. Interrupts : Using the `attachInterrupt()` function the external interrupts are used to lower the trigger whenever the pin is LOW at Pin 2 and 3.

LED : Built-in LED is connected to Pin 13 and turns ON or OFF according to the HIGH and LOW value provided in the code.

I2C : A4 (SDA) and A5 (SCL) support the I2C communication.

AREF : Analog Pins uses the Reference Voltage with the command `analogReference()`.

Reset : To reset the Microcontroller.

3.2 Ultrasonic Sensor

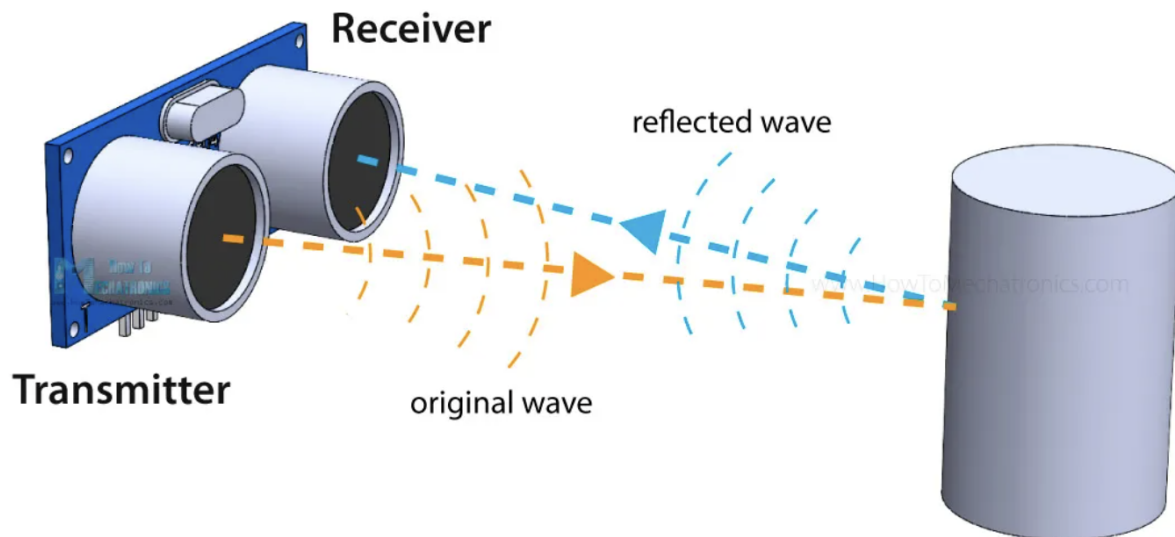


Figure 5: Ultrasonics waves transmitted and received by an Ultrasonic Sensor

The HC-SR04 Ultrasonic sensor is used to measure the distance by emitting an Ultrasound frequency of 40kHz. If any object is detected on the path, the Ultrasound waves bounce back to the receiver end as shown in Figure 5. The distance of the object is calculated by the halving time required for the Ultrasound waves to detect the object and bounce back to the receiver. It is calculated using the formula below;

$$Distance = \frac{Speed\ of\ sound\ in\ air \times Time\ from\ transmitter\ to\ the\ receiver}{2}$$

For example: Find the distance of the object if the time required to detect the object is 150 μ s.

Given: Speed of Sound in Air = 340 m/s = 0.034cm/ μ s

$$\begin{aligned}\text{Time} &= 150 \mu\text{s} \\ \text{Distance} &= \frac{\text{Speed} \times \text{Time}}{2} \\ &= \frac{0.034 \times 150}{2} \\ &= 2.55 \text{ cm}\end{aligned}$$

General Pin Representation of Ultrasonic Sensor:

- Pin 1 - Vcc
- Pin 2 - Trigger
- Pin 3 - Echo
- Pin 4 - Ground

In the current project, we're using 3 Ultrasonic sensors to detect the object in 3 different directions(Right, Front and Left). The Trigger and Echo of the Front Ultrasonic sensor are connected to the Pin2 and 3 respectively. The Trigger and Echo of the Left Ultrasonic sensor are connected to the Pin5 and 4 respectively. Similarly, the trigger and Echo of the Right Ultrasonic sensor are connected to Pin6 and 7 respectively.

3.3 Pololu IR sensor Array

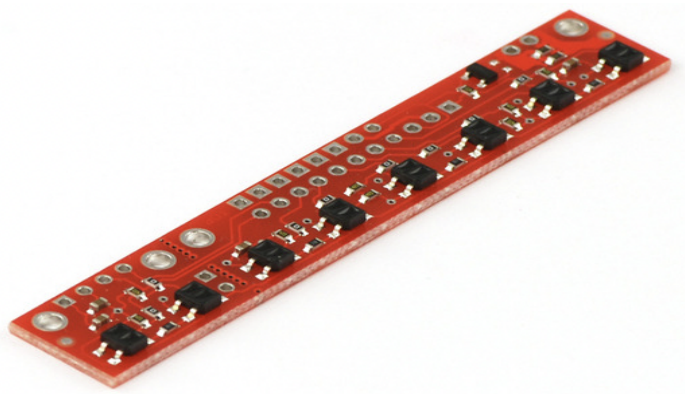


Figure 6: QTR-8RC Reflectance Sensor Array

With an in-built 8 IR LED, the IR sensor array is mostly utilized for sensing light and dark points on a plane, which just so happen to be fantastic for line following robots. Pairing up with LEDs for low current consumption and MOSFET for power savings are some of the significant features of QTR-8RC. All 8 sensors have separate digital I/O measurable outputs. Apart from that, it doesn't require any analog to digital converter.

Specifications

- Dimensions: 2.95" x 0.5" x 0.125" (74.93mm x 12.7mm x 3.175mm)
- Required operating Voltage : 3.3-5.0 V
- Required supply current: 100 mA
- Output format: 8 digital I/O-compatible signals
- Optimal sensing distance: 0.125" (3 mm)

In the current project, we're using the Pololu IR sensor array to detect the black line (tape) and follow the robot on the track given. First calibration of the IR sensor takes place which may require 7-10 seconds. Each IR returns a value of 0 for maximum reflectiveness and 1000 for minimum reflectiveness. Each IR LED is passed over the black tape for a split second and returned to the white paper background briefly, this is repeated as many times as possible for better calibration. The Pin representation of the Pololu sensor's connection to the Arduino Nano board was explained in the previous chapter.

4. Software Session

4.1 Code:

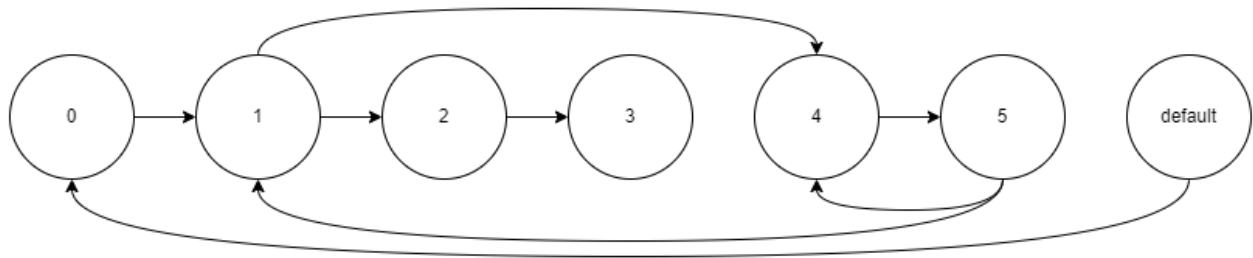


Figure 7: State Machine Diagram showing the flow of the program

The libraries for the Servo and Reflectance Sensor are included in the code, these libraries make it easier to run the Servos and Reflectance sensor.

The pins connected to electronic devices and the necessary global variables are initialized.

The Code is broken down into functions and there's a brief description of each function below:

Void setup(); there's a calibration sequence, this sequence is used to make the robot auto-calibrate the Reflectance sensor by itself, the calibration process takes about 7-10 seconds.

Void loop(); "switch... case" was used to control the flow and branching of the program;

- Case 0; This case statement is used to make the robot get from the Home (H1 or H2) position to intersection i0.
- Case 1; This case statement allows the robot to get from intersection i0 to intersection i5 and uses the Left and Right Ultrasonic sensors to check for objects from A1 to A5 and B1 to B5 while it travels to the final intersection.
- Case 2; This case statement gets triggered when the robot gets to the intersection i5, the front ultrasonic sensor is turned on and used to check for objects in location C1.
- Case 3; This case statement stops the robot and flashes the LED to mark the end of the operation.
- Case 4; This Case statement branches the robot off the main track when the left or/and right ultrasonic sensor detects an object.
- Case 5; This case statement returns the robot back to the main track or makes the robot go to the second object if there are multiple objects at an intersection.

`readUltraSonic()`; this function receives 2 parameters, one for the trigger pin and the other for the echo pin. The function takes these parameters and sends a pulse to the trigger pin and checks to measure the duration of the return pulse on the echo pin and calculates the distance with the time, with the distance being returned.

`flashSequence()`; this function takes 1 parameter and uses a for loop to blink the LEDs a number of times determined by the parameter, no returned value.

`intersectionSequence()`; this function takes no parameters and it's used to blink the LEDs in a different pattern on getting to an intersection.

`objSequence()`; this function takes no parameters and it's used to flash the LEDs in a different pattern on stopping at a certain distance from the object.

`Stop()`; this function stops the servo motors.

`moveForward()`; this function takes one parameter and moves the robot forward.

`moveBackward()`; this function takes one parameter and moves the robot in reverse.

`turnLeft()`; this function takes one parameter and makes the robot spin left on the wheel axis, by making the wheels spin in opposite directions.

`turnRight()`; this function takes one parameter and makes the robot spin right on the wheel axis, by making the wheels spin in opposite directions.

5. Result and Conclusion

After the implementation of the line following robot, 5 objects were kept at different locations of the intersection points. In our demonstration video there was one more object kept at intersection i0 to show that the robot doesn't check for objects at intersection i0. The robot keeps following the designed track and stops after coming to the final location C1 while checking for objects at the various intersections. By using different delay values the robot was allowed to move and stop at certain points.

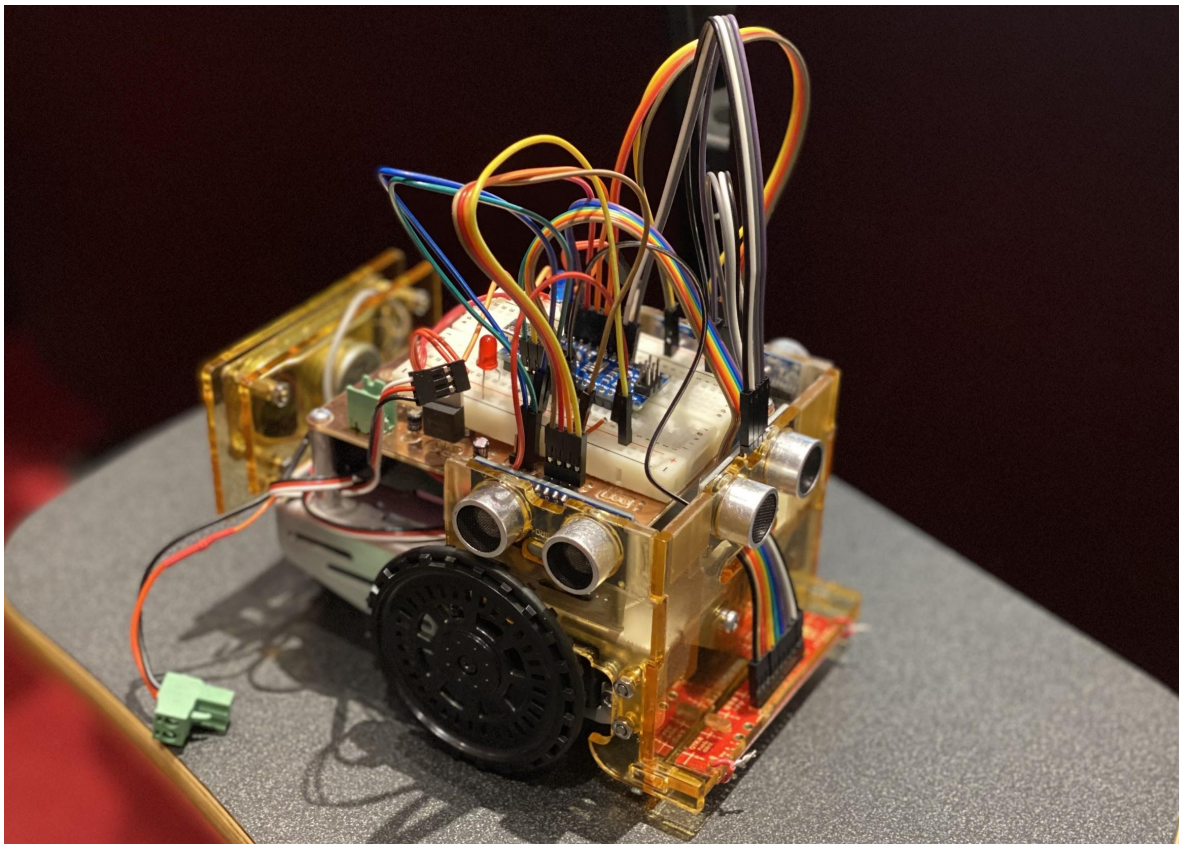


Figure 8: Picture of the Line Following Robot